

Phase 2: 3D Engine Transition

Replacing Phaser 3 with Babylon.js, Room Loading, Camera System, and Basic 3D Avatar

Author: Daniel | **Date:** Friday, 18 September 2026 | **Series:** HavenWorld Developer Build Guide | **Part:** 5A of Phase 2 | **License:** GPL-3.0 (project) / Apache-2.0 (Babylon.js) | **Repo:** github.com/dboone323/HavenWorld

Table of Contents

1. [Section 1 — Project Context and Phase Recap](#)
2. [Section 2 — Why 3D Now? Architecture Decision](#)
3. [Section 3 — Prerequisites and Tool Versions](#)
4. [Section 4 — Removing Phaser 3 from the Client](#)
5. [Section 5 — Installing Babylon.js](#)
6. [Section 6 — Project Structure After Transition](#)
7. [Section 7 — HavenEngine.ts: Babylon.js Engine Singleton](#)
8. [Section 8 — SceneManager.ts: Scene Registry and Transitions](#)
9. [Section 9 — RoomLoader.ts: Loading 3D Rooms from GLB](#)
10. [Section 10 — RoomScene.ts: The Main 3D Room](#)
11. [Section 11 — AvatarController.ts: Local Player 3D Avatar](#)
12. [Section 12 — RemoteAvatar.ts: Other Players' Avatars](#)
13. [Section 13 — InputController.ts: Click-to-Move Input](#)
14. [Section 14 — ChatOverlay.ts: HTML Chat System](#)
15. [Section 15 — Updated main.ts Entry Point](#)
16. [Section 16 — Updated index.html](#)
17. [Section 17 — Updating packages/shared for 3D](#)
18. [Section 18 — Updating the Server for 3D Positions](#)
19. [Section 19 — Asset Placeholders: Getting Started Without 3D Art](#)
20. [Section 20 — Running and Testing Phase 2 \(Part A\)](#)
21. [Section 21 — CI/CD Notes](#)
22. [Section 22 — Troubleshooting Phase 2A](#)

1 — Project Context and Phase Recap

HavenWorld is a browser-based social virtual world game built entirely on free, open-source infrastructure. It is licensed under **GPL-3.0**, uses **pnpm workspaces** as its monorepo manager, and is deployed at zero cost using Cloudflare Pages (frontend) and Oracle Cloud Always-Free VMs (backend). No paid services are ever used.

The monorepo lives at github.com/dboone323/HavenWorld and is organized into three workspace packages:

- **apps/client** — The browser frontend. Was Vite 6 + Phaser 3. *This phase replaces Phaser with Babylon.js.*
- **apps/server** — Express 5 + Socket.io 4 + Prisma + PostgreSQL 16 + Redis 7. *Unchanged in Part 5A.*
- **packages/shared** — TypeScript interfaces and Socket.io event constants shared between client and server. *Minor additions for 3D position types only.*

Deployment URLs:

- Frontend: <https://havenworld.pages.dev> (Cloudflare Pages)
- API/WebSocket: [https://\[oracle-ip\].nip.io](https://[oracle-ip].nip.io) (Oracle Cloud Always-Free VM)

1.1 — What Prior Phases Delivered

Phase	Parts	Key Deliverables
Phase 0	Parts 1-2	Mac dev environment, Oracle Cloud VM, pnpm monorepo scaffold, GitHub Actions CI/CD, Cloudflare Pages deployment pipeline

Phase	Parts	Key Deliverables
Phase 1	Parts 3–4C	Phaser 3 client with 5 scenes (Boot, Login, Lobby, Room, UI), 9-layer 2D sprite avatar system, Tiled tilemap rooms, full Express 5 server, Prisma schema with 10 tables, Socket.io real-time player sync, JWT auth flow, Redis session cache

1.2 — Phase 2 Goal

The goal of Phase 2 is to **replace the 2D Phaser 3 client with a full 3D Babylon.js client** while keeping the existing Express server, Prisma schema, Socket.io event architecture, and auth system *completely unchanged*. The 3D client must connect to the same WebSocket server, use the same JWT tokens, and emit/receive the same Socket.io events — with only the position payload upgrading from 2D tile coordinates to 3D world-space meters.

Part 5A covers the engine setup, scene infrastructure, camera, input, chat overlay, and placeholder geometry. Part 5B (coming next) will cover the Blender pipeline, real rigged avatars, Mixamo animations, morph targets, and the furniture placement system.



Part 5A Strategy

Build and test with placeholder geometry (Babylon MeshBuilder primitives) first. Once the engine, socket sync, and input systems all work, swap in real GLB assets in Part 5B. This keeps the feedback loop short and the debug surface small.

2 — Why 3D Now? Architecture Decision

The Phaser 3 client served Phase 1 well — rapid prototyping, familiar 2D tilemap tooling, and a lightweight bundle. But the HavenWorld roadmap calls for **room furniture placement, avatar customization with morph targets, dynamic lighting for ambiance, and eventual WebXR support**. Those requirements make a real 3D engine necessary, not optional. The question is *which* one.

2.1 — Engine Comparison

Criterion	Three.js	Babylon.js	PlayCanvas
License	MIT	Apache-2.0	MIT (engine) / cloud editor proprietary
Core bundle size	~168 KB min+gzip	~1.4 MB min (tree-shakeable)	~300 KB min+gzip
Physics built-in	No (use Cannon.js / Rapier externally)	Yes — Havok (WASM) + Cannon.js + Ammo.js	Yes — Ammo.js (Bullet)
Visual editor	None official	Free web Playground + Inspector + NME	Cloud editor (proprietary, free tier limited)
WebGPU support	Yes (r162+)	Yes (7.x native)	Yes (1.x beta)
TypeScript-first	Types via @types/three	100% TypeScript source	Types included
Scene Inspector	No (use third-party)	Built-in (Ctrl+I, zero config)	In cloud editor only
Social world precedent	Limited	Proven in Mizzou capstone WebXR social world with Socket.io	Limited open-source examples
GPL-3.0 compatible	Yes (MIT)	Yes (Apache-2.0 is compatible as a dependency)	Partially (engine yes, cloud editor no)
Best for HavenWorld	No	Yes — Winner	No

Babylon.js wins for HavenWorld for four concrete reasons: (1) its Apache-2.0 license is legally compatible as a dependency in a GPL-3.0 project; (2) it is 100% TypeScript source with no @types package needed; (3) the built-in Inspector removes an entire category of debug tooling overhead; and (4) it has documented, working examples of real-time multiplayer social worlds using Socket.io — specifically the University of Missouri WebXR capstone project, which shares almost identical architecture to HavenWorld.

2.2 — Camera Philosophy: Isometric-Style Fixed Angle

HavenWorld does *not* use a free-roam first-person camera. Instead, it uses a **fixed-angle isometric-style camera** — a real 3D `ArcRotateCamera` locked to approximately 45 degrees downward, with rotation disabled for the end user. This is the same approach used by Club Penguin 3D, YoWorld, and Avakin Life: the world is genuinely 3D (geometry, lighting, shadows, depth), but the camera angle is curated to maintain the *cozy, readable social game aesthetic* where you can always see your avatar and the room layout clearly. Users zoom in/out with the scroll wheel, but cannot orbit. This also dramatically simplifies pathfinding and click-to-move hit detection.

3 — Prerequisites and Tool Versions

The following tools must be installed and at the specified versions before beginning Phase 2. Tools marked as already installed from prior phases are noted.

Tool	Required Version	Status	Notes
Node.js	22 LTS	Already installed (Phase 0)	Verify: <code>node -v</code>
pnpm	9.x	Already installed (Phase 0)	Verify: <code>pnpm -v</code>
Blender	5.1 LTS	New — install now	Free, blender.org. Used in Part 5B for asset authoring.
glTF-transform CLI	latest	New — install now	GLB optimization + Draco

Tool	Required Version	Status	Notes
			compression
@babylonjs/core	7.x	New — Section 5	Main engine package
@babylonjs/loaders	7.x	New — Section 5	GLB/GLTF import support
@babylonjs/materials	7.x	New — Section 5	PBR, node, water materials
@babylonjs/havok	7.x	New — Section 5	Havok physics WASM
@babylonjs/inspector	7.x	New — Section 5 (devDep)	Scene debugger, Ctrl+I in dev

3.1 — Installing Blender 5.1 LTS

MAC Install via Homebrew (recommended for automatic updates):

```
brew install --cask blender
```

MAC Or download directly from blender.org/download/lts/ — choose the macOS Apple Silicon or Intel dmg, mount it, and drag Blender to your Applications folder. Verify the install:

```
blender --version # Expected: Blender 5.1.x
```

LINUX Install via Snap on Ubuntu/Debian-based systems:

```
sudo snap install blender --classic blender --version
```

3.2 — Installing gltf-transform CLI

MAC **LINUX** Install globally via npm (not pnpm, to keep it as a global system tool):

```
npm install -g @gltf-transform/cli gltf-transform --version
```

You will use gltf-transform in Part 5B to apply **Draco mesh compression** and **KTX2 texture compression** to exported GLB files before committing them to the repo. In Part 5A, placeholders require no compression.

3.3 — Babylon.js Inspector

The **Babylon.js Inspector** is a free, built-in scene debugging panel loaded dynamically in development builds. Once active, press **Ctrl+I** (or **Cmd+I** on Mac) to toggle it. It shows the full scene graph, mesh properties, material channels,

animation groups, physics bodies, and performance metrics — the equivalent of having a full game engine editor inside the browser. It is excluded from production builds via dynamic import.

4 — Removing Phaser 3 from the Client

Before installing Babylon.js, cleanly remove all Phaser 3 artifacts. This avoids bundle conflicts and keeps the dependency graph clean.

4.1 — Uninstall Phaser Packages

```
cd ~/projects/HavenWorld pnpm remove phaser easystarjs --filter client
```

Verify the removal worked — `phaser` and `easystarjs` should no longer appear in `apps/client/package.json` under `dependencies`.

4.2 — Delete Old Phaser Scene Files

```
rm apps/client/src/scenes/Boot.ts rm apps/client/src/scenes/Login.ts rm  
apps/client/src/scenes/Lobby.ts rm apps/client/src/scenes/Room.ts rm  
apps/client/src/scenes/UIScene.ts
```

4.3 — Delete Old 2D Avatar Sprite Sheets

```
rm -rf apps/client/src/assets/avatars/
```

⚠ Keep These Files

Do NOT delete `src/services/AuthService.ts` or `src/services/SocketService.ts`. These are reused entirely unchanged. Also keep `src/index.html` (you will update it in Section 16, not replace it) and `src/style.css`.

4.4 — What packages/shared Keeps vs. Changes

The Socket.io event name constants (`PLAYER_MOVE`, `CHAT_SEND`, `AVATAR_UPDATE`, etc.) defined in `packages/shared` are **completely unchanged**. The only addition is a new `PlayerPosition3D` interface to replace the 2D `{x, y}` tile-coordinate payload with `{x, y, z, rotY}` world-space meters. See Section 17 for the exact diff.

5 — Installing Babylon.js

5.1 — Install Commands

```
pnpm add @babylonjs/core @babylonjs/loaders @babylonjs/materials  
@babylonjs/havok --filter client pnpm add -D @babylonjs/inspector --filter  
client
```

After installation, verify `apps/client/package.json` contains the Babylon packages in dependencies and `@babylonjs/inspector` in devDependencies. All four runtime packages should be at the same 7.x minor version to avoid peer dependency conflicts.

5.2 — Updated vite.config.ts

Babylon.js ships a **Havok physics WASM binary** (`HavokPhysics.js.wasm`). Vite must be told not to inline this file and not to try to bundle it through Rollup's module graph. The `optimizeDeps.exclude` directive and `assetsInlineLimit: 0` handle this.

Replace the entire `apps/client/vite.config.ts` with:

```
import { defineConfig } from 'vite' export default  
defineConfig({ optimizeDeps: { exclude: ['@babylonjs/havok'], },  
assetsInlineLimit: 0, server: { port: 5173, }, build:  
{ rollupOptions: { output: { manualChunks:  
{ babylon: [  
'@babylonjs/core',  
'@babylonjs/loaders', '@babylonjs/materials',  
, }, }, }, }, }, })
```

Why exclude @babylonjs/havok?

`@babylonjs/havok` loads its own WASM file at runtime using a relative URL. If Vite pre-bundles it, the WASM path breaks. Excluding it from `optimizeDeps` makes Vite serve it as-is from `node_modules`, preserving the runtime URL resolution. The `manualChunks` config splits the large Babylon bundle from your app code, improving cache hit rates on subsequent deploys.

6 — Project Structure After Transition

After removing Phaser and adding Babylon.js, the `apps/client/src/` directory is reorganized into domain-driven folders. Here is the complete new structure:

```

src/ └─ engine/ └─ HavenEngine.ts          # Babylon.js engine singleton
wrapper └─ SceneManager.ts                # Scene registry and transitions
└─ InputController.ts                    # Keyboard + mouse + touch input └─ scenes/
└─ LoginScene.ts                        # Auth UI overlay (HTML, not Babylon) └─
LobbyScene.ts                          # Room select screen └─ RoomScene.ts      #
Main 3D room (Babylon.js) └─ world/ └─ RoomLoader.ts          # GLB
room file loader └─ AvatarController.ts  # Local player avatar └─
RemoteAvatar.ts      # Other players' avatars └─ FurnitureManager.ts
# Room furniture placement (Part 5B) └─ Pathfinder.ts          # 3D nav
mesh pathfinding (Part 5B) └─ ui/ └─ ChatOverlay.ts            # Chat box
(HTML overlay) └─ AvatarCustomizer.ts    # Avatar editor UI (Part 5B) └─
└─ HUD.ts                          # Heads-up display elements └─ services/ └─
AuthService.ts                    # Unchanged from Phase 1 └─ SocketService.ts
# Unchanged from Phase 1 └─ main.ts          # Entry point

```

The `services/` folder contents are **carried over unchanged** from Phase 1. The `engine/` and `world/` folders are entirely new. The `scenes/` folder is rebuilt from scratch — the new scene files are Babylon-native TypeScript classes, not Phaser Scene subclasses. Create all directories now:

```
mkdir -p apps/client/src/{engine,scenes,world,ui}
```

7 — HavenEngine.ts: Babylon.js Engine Singleton

HavenEngine is the central orchestrator of the Babylon.js runtime. It wraps the Engine (or WebGPUEngine in future) instance, manages the render loop, handles canvas resize, and optionally loads the Inspector in development mode. Create the file at `apps/client/src/engine/HavenEngine.ts`:

```

import { Engine, Scene, HavokPlugin, } from '@babylonjs/core' import
HavokPhysics from '@babylonjs/havok' export class HavenEngine { private
static _instance: HavenEngine | null = null private _engine: Engine
private _canvas: HTMLCanvasElement private _activeScene: Scene | null =
null private _inspectorLoaded = false private constructor(canvas:
HTMLCanvasElement) { this._canvas = canvas this._engine = new
Engine(canvas, true, { preserveDrawingBuffer: true, stencil:
true, disableWebGL2Support: false, }) // Start the render loop
- renders whatever _activeScene is set to this._engine.runRenderLoop(()
=> { if (this._activeScene && this._activeScene.activeCamera) {
this._activeScene.render() } }) // Resize handler
window.addEventListener('resize', this._onResize) // Cleanup on unload
window.addEventListener('beforeunload', this._onUnload)
console.log(`[HavenEngine] Babylon.js ${Engine.Version} initialized.`) }
/** Singleton accessor. Call HavenEngine.getInstance() to get or create. */
public static async getInstance(canvas?: HTMLCanvasElement): Promise { if
(!HavenEngine._instance) { if (!canvas) throw new Error('[HavenEngine]
Canvas required for first initialization.') HavenEngine._instance = new
HavenEngine(canvas) await HavenEngine._instance._initPhysics() if
(import.meta.env.DEV) { await HavenEngine._instance._loadInspector()

```

```

}    }    return HavenEngine._instance }    /** Initialize Havok physics
plugin and attach to global scope for scene use. */    private async
_initPhysics(): Promise {    try {        const havokInstance = await
HavokPhysics()        const plugin = new HavokPlugin(true, havokInstance)
// Store plugin so scenes can enable physics: scene.enablePhysics(gravity,
plugin)        ;(window as unknown as Record).__havenHavokPlugin = plugin
console.log('[HavenEngine] Havok physics initialized.')    } catch (err) {
console.warn('[HavenEngine] Havok physics failed to load. Physics disabled.',
err)    }    }    /** Dynamically import the Inspector in dev mode only –
excluded from prod bundle. */    private async _loadInspector(): Promise {
if (this._inspectorLoaded) return    try {        await
import('@babylonjs/inspector')        this._inspectorLoaded = true
console.log('[HavenEngine] Inspector loaded. Press Ctrl+I (or Cmd+I) to
toggle.')    } catch (err) {        console.warn('[HavenEngine] Inspector
could not be loaded:', err)    }    }    /** Toggle Babylon Inspector
visibility. Only works in dev (inspector must be loaded). */    public
toggleInspector(): void {        if (!this._activeScene) return        if
(this._activeScene.debugLayer.isVisible())
{            this._activeScene.debugLayer.hide()        } else
{            this._activeScene.debugLayer.show({ embedMode:
true })        }    }    /** Set the active scene to be rendered each frame. */
public setActiveScene(scene: Scene): void {        this._activeScene = scene    }
public get engine(): Engine {        return this._engine    }    public get
canvas(): HTMLCanvasElement {        return this._canvas    }    public get
activeScene(): Scene | null {        return this._activeScene    }    private
_onResize = (): void => {        this._engine.resize()    }    private _onUnload
= (): void => {        this.dispose()    }    public dispose(): void
{        window.removeEventListener('resize', this._onResize)
window.removeEventListener('beforeunload', this._onUnload)
this._activeScene?.dispose()        this._engine.dispose()
HavenEngine._instance = null        console.log('[HavenEngine] Disposed.')    } }

```

8 — SceneManager.ts: Scene Registry and Transitions

The **SceneManager** maintains a registry of named scene factories and handles the lifecycle of switching between them — disposing the old scene before activating the new one. Create `apps/client/src/engine/SceneManager.ts`:

```

import { Scene } from '@babylonjs/core' import { HavenEngine } from
'./HavenEngine' type SceneFactory = (engine: HavenEngine) => Promise export
class SceneManager {    private static _instance: SceneManager | null = null
private _registry: Map = new Map()    private _currentSceneName: string | null
= null    private _currentScene: Scene | null = null    private constructor()
{}    public static getInstance(): SceneManager {        if (!
SceneManager._instance) {            SceneManager._instance = new SceneManager()
}        return SceneManager._instance    }    /**    * Register a named scene
factory function.    * The factory receives the HavenEngine instance and
returns a fully-initialized Scene.    */    public register(name: string,
factory: SceneFactory): void {        this._registry.set(name, factory)
console.log(`[SceneManager] Registered scene: "${name}"`)    }    /**    *
Switch to a named scene. Disposes the current scene first, then    * runs the
factory, sets the new scene active on the engine, and returns it.    */

```

```

public async switchTo(name: string): Promise {    const factory =
this._registry.get(name)    if (!factory) throw new Error(`[SceneManager]
Unknown scene: "${name}"`)    // Dispose old scene    if
(this._currentScene) {    console.log(`[SceneManager] Disposing scene: "$
{this._currentSceneName}"`)    this._currentScene.dispose()
this._currentScene = null    }    const haven = await
HavenEngine.getInstance()    console.log(`[SceneManager] Loading scene: "$
{name}"`)    const scene = await factory(haven)
haven.setActiveScene(scene)    this._currentScene = scene
this._currentSceneName = name    console.log(`[SceneManager] Active scene:
"${name}"`)    return scene    }    public get currentSceneName(): string |
null {    return this._currentSceneName    }    public get currentScene():
Scene | null {    return this._currentScene    } }

```

9 — RoomLoader.ts: Loading 3D Rooms from GLB

The **RoomLoader** is responsible for fetching a GLB file from `/assets/rooms/{roomId}.glb`, importing it into the active Babylon.js scene, identifying walkable and navigation surfaces by mesh name convention, and returning structured references to the loaded geometry. Create `apps/client/src/world/RoomLoader.ts`:

```

import { Scene, AbstractMesh, SceneLoader, Vector3, Tags, } from
'@babylonjs/core' import '@babylonjs/loaders/glTF' export interface
RoomLoadResult { rootMesh: AbstractMesh walkableMeshes: AbstractMesh[]
navMeshes: AbstractMesh[] collisionMeshes: AbstractMesh[] allMeshes:
AbstractMesh[] } export class RoomLoader { private _scene: Scene
constructor(scene: Scene) { this._scene = scene } /** * Load a
room GLB by ID from /assets/rooms/{roomId}.glb. * Meshes are categorized
by name prefix convention: * - "NavMesh_*" → navigation surface
(invisible) * - "Walkable_*" → floor / walkable area (visible, tagged
"walkable") * - "Collision_*" → invisible collision geometry * -
All others → visible room geometry */ public async
load(roomId: string, onProgress?: (event: ProgressEvent) =>
void ): Promise { const url = `/assets/rooms/${roomId}.glb`
console.log(`[RoomLoader] Loading room: ${url}`) const result = await
SceneLoader.ImportMeshAsync('', '', // mesh names – empty
string loads all '', // root URL – empty because we use full
path url, this._scene, onProgress ) const
allMeshes = result.meshes as AbstractMesh[] const rootMesh = allMeshes[0]
const walkableMeshes: AbstractMesh[] = [] const navMeshes: AbstractMesh[]
= [] const collisionMeshes: AbstractMesh[] = [] for (const mesh of
allMeshes) { const name = mesh.name if
(name.startsWith('NavMesh_')) { mesh.isVisible = false
mesh.isPickable = false Tags.AddTagsTo(mesh, 'navmesh')
navMeshes.push(mesh) } else if (name.startsWith('Walkable_') || name
=== 'Floor') { Tags.AddTagsTo(mesh, 'walkable')
mesh.isPickable = true walkableMeshes.push(mesh) } else if
(name.startsWith('Collision_')) { mesh.isVisible = false
mesh.isPickable = false Tags.AddTagsTo(mesh, 'collision')

```

```
collisionMeshes.push(mesh) } else { // Standard room geometry
- visible but not directly pickable for movement mesh.isPickable =
false } } console.log( `[RoomLoader] Room "${roomId}"
loaded. ` + `Meshes: ${allMeshes.length}, Walkable: $
{walkableMeshes.length}, ` + `NavMesh: ${navMeshes.length}, Collision:
${collisionMeshes.length}` ) return { rootMesh, walkableMeshes,
navMeshes, collisionMeshes, allMeshes } } /** * Unload the current
room by disposing all tagged meshes. * Called by SceneManager before
loading a new room. */ public unload(): void { const tagged =
this._scene.getMeshesByTags('walkable navmesh collision')
tagged.forEach(m => m.dispose()) } }
```

RecastJS Navigation Note

Full crowd navigation using Babylon's built-in **RecastJS** integration (RecastJSPlugin) is implemented in Part 5B after real GLB rooms with proper nav mesh geometry are available. In Part 5A, the AvatarController uses direct vector movement (no pathfinding) over the placeholder flat floor, which is sufficient to verify socket sync and animation transitions.

10 — RoomScene.ts: The Main 3D Room

The **RoomScene** is the primary Babylon.js scene. It sets up the camera, lights, loads the room geometry, creates the local avatar, wires up all Socket.io events for multiplayer, and handles click-to-move input. Create

apps/client/src/scenes/RoomScene.ts:

```
import { Scene, ArcRotateCamera, HemisphericLight, DirectionalLight,
Vector3, Color3, ShadowGenerator, KeyboardEventTypes, } from
'@babylonjs/core' import { HavenEngine } from '../engine/HavenEngine' import
{ RoomLoader } from '../world/RoomLoader' import { AvatarController } from
'../world/AvatarController' import { RemoteAvatar } from
'../world/RemoteAvatar' import { InputController } from
'../engine/InputController' import { SocketService } from
'../services/SocketService' import { AuthService } from
'../services/AuthService' export async function createRoomScene( haven:
HavenEngine, roomId: string ): Promise { const scene = new
Scene(haven.engine) // — CAMERA
//
ArcRotateCamera: alpha = azimuth, beta = elevation, radius = distance //
Fixed isometric-style: 45° down, locked rotation — no user orbit. const
camera = new ArcRotateCamera( 'isoCam', -Math.PI / 4, // alpha: 45°
horizontal offset (south-west view) Math.PI / 4, // beta: 45° downward
angle 18, // radius: ~18 meters from target
Vector3.Zero(), scene ) camera.lowerRadiusLimit = 8
```

```

camera.upperRadiusLimit = 28    camera.lowerBetaLimit = Math.PI / 4    // Lock
to exactly 45° down    camera.upperBetaLimit = Math.PI / 4    // Same –
prevents vertical orbit    camera.lowerAlphaLimit = camera.alpha // Lock
horizontal rotation    camera.upperAlphaLimit = camera.alpha // Lock
horizontal rotation    // Allow only zoom (wheel/pinch), disable rotation
input    camera.inputs.removeByType('ArcRotateCameraKeyboardMoveInput')
camera.inputs.removeByType('ArcRotateCameraPointersInput')    // Re-attach
only mouse wheel zoom    camera.attachControl(haven.canvas, false)    // —
LIGHTS    _____    const
ambient = new HemisphericLight('ambient', new Vector3(0, 1, 0), scene)
ambient.intensity = 0.6    ambient.diffuse = new Color3(1, 0.98, 0.92)
ambientgroundColor = new Color3(0.2, 0.2, 0.3)    const sun = new
DirectionalLight('sun', new Vector3(-1, -2, -1), scene)    sun.intensity = 0.8
sun.position = new Vector3(10, 20, 10)    const shadowGen = new
ShadowGenerator(1024, sun)    shadowGen.useBlurExponentialShadowMap =
true    // — ROOM GEOMETRY    _____

RoomLoader(scene)    let roomLoaded = false    try {    const loader = new
await loader.load(roomId, (evt) => {    if (evt.lengthComputable) {
const pct = Math.round((evt.loaded / evt.total) * 100)
console.log(`[RoomScene] Loading room: ${pct}%`)    }    })
roomLoaded = true    console.log('[RoomScene] Room GLB loaded.')    //
Cast shadows on walkable surfaces    for (const mesh of
roomResult.walkableMeshes) {    mesh.receiveShadows = true    }    } catch
{    console.warn('[RoomScene] GLB room not found – falling back to
placeholder geometry.')    // Placeholder room geometry (Section 19.1) is
created inline:    const { createPlaceholderRoom } = await
import('../world/PlaceholderRoom')    createPlaceholderRoom(scene)    }    //
— LOCAL AVATAR    _____
const me = AuthService.getCurrentUser()    const localAvatar = new
AvatarController(scene, me)    await localAvatar.init()
shadowGen.addShadowCaster(localAvatar.rootMesh)    // Camera follows local
avatar    camera.setTarget(localAvatar.rootMesh.position)
scene.registerBeforeRender(() => {    camera.target =
Vector3.Lerp(camera.target, localAvatar.rootMesh.position, 0.08)    })    //
— REMOTE AVATARS    _____
const remoteAvatars = new Map()    const socket = SocketService.getInstance()
socket.on('player:join', async (data: { userId: string; username: string;
position: { x: number; y: number; z: number; rotY: number }; avatarData:
unknown }) => {    if (data.userId === me.id) return    const remote = new
RemoteAvatar(scene, data.userId, data.username)    await remote.init()
remote.setPosition(new Vector3(data.position.x, data.position.y,
data.position.z))    remoteAvatars.set(data.userId, remote)
shadowGen.addShadowCaster(remote.rootMesh)    })    socket.on('player:move',
(data: { userId: string; x: number; y: number; z: number; rotY: number }) =>
{    if (data.userId === me.id) return    const remote =
remoteAvatars.get(data.userId)    if (remote) {    remote.moveTo(new
Vector3(data.x, data.y, data.z), data.rotY)    }    })
socket.on('avatar:update', (data: { userId: string; avatarData: unknown }) =>
{    const remote = remoteAvatars.get(data.userId)    if (remote)
remote.applyAvatarData(data.avatarData)    })    socket.on('chat:message',
(data: { userId: string; username: string; message: string }) => {    const
remote = remoteAvatars.get(data.userId)    if (remote)
remote.showChatBubble(data.message)    })    socket.on('player:leave', (data:
{ userId: string }) => {    const remote = remoteAvatars.get(data.userId)
if (remote) {    remote.dispose()    remoteAvatars.delete(data.userId)
}    })    // — INPUT    _____
= new InputController(scene, localAvatar)    input.enable()    // — DEV
INSPECTOR SHORTCUT    _____    if

```

```
(import.meta.env.DEV) {      scene.onKeyboardObservable.add((kbInfo) => {
if (      kbInfo.type === KeyboardEventTypes.KEYDOWN &&
kbInfo.event.key === 'I' &&      (kbInfo.event.ctrlKey ||
kbInfo.event.metaKey)      ) {      kbInfo.event.preventDefault()
haven.toggleInspector()      }      })      }      // Tell server we've joined
socket.emit('room:join', { roomId })      return scene }
```

11 — AvatarController.ts: Local Player 3D Avatar

The **AvatarController** manages the local player's avatar mesh, animations, movement, and name tag. Create `apps/client/src/world/AvatarController.ts`:

```
import { Scene, TransformNode, AbstractMesh, AnimationGroup,
Vector3, Color3, StandardMaterial, MeshBuilder, SceneLoader,
BillboardMode, } from '@babylonjs/core' import { AdvancedDynamicTexture,
TextBlock } from '@babylonjs/gui' import { SocketService } from
'../services/SocketService' export interface AvatarData { skinColor:
string hairColor: string eyeColor: string bodyMorphs?: Record } export
interface UserProfile { id: string username: string avatarData:
AvatarData } export class AvatarController { public rootMesh!:
TransformNode private _scene: Scene private _user: UserProfile private
_meshes: AbstractMesh[] = [] private _animations: Map = new Map() private
_currentAnim: AnimationGroup | null = null private _targetPosition: Vector3
| null = null private _isMoving = false private _moveSpeed = 3.5 //
meters per second private _nameTagMesh: AbstractMesh | null = null
constructor(scene: Scene, user: UserProfile) { this._scene = scene
this._user = user } public async init(): Promise { try { await
this._loadGLB() } catch { console.warn('[AvatarController] GLB not
found – using placeholder capsule.') this._buildPlaceholder() }
this._buildNameTag() this._registerRenderLoop() } private async
_loadGLB(): Promise { const result = await SceneLoader.ImportMeshAsync(
'', '', '/assets/avatars/base_avatar.glb',
this._scene ) this.rootMesh = new TransformNode('localAvatar_root',
this._scene) result.meshes[0].parent = this.rootMesh this._meshes =
result.meshes as AbstractMesh[] // Apply avatar customization
this._applyMaterials(this._user.avatarData) // Register animation groups
by name for (const ag of result.animationGroups)
{ this._animations.set(ag.name, ag) ag.stop() }
this._playAnimation('idle', true) } private _buildPlaceholder(): void {
this.rootMesh = new TransformNode('localAvatar_root', this._scene) const
body = MeshBuilder.CreateCapsule( 'avatar_body', { radius: 0.25,
height: 1.7, tessellation: 8 }, this._scene ) body.parent =
this.rootMesh body.position.y = 0.85 const head =
MeshBuilder.CreateSphere( 'avatar_head', { diameter: 0.4,
segments: 8 }, this._scene ) head.parent = this.rootMesh
head.position.y = 1.95 const mat = new StandardMaterial('avatar_mat',
this._scene) mat.diffuseColor =
Color3.FromHexString(this._user.avatarData.skinColor || '#F5CBA7')
body.material = mat head.material = mat this._meshes = [body, head]
} private _applyMaterials(data: AvatarData): void { // Apply morph
targets for body shape if (data.bodyMorphs) { for (const [name,
value] of Object.entries(data.bodyMorphs)) { for (const mesh of
this._meshes) { const manager = (mesh as unknown as
```

```

{ morphTargetManager?: { numTargets: number; getTarget: (i: number) =>
{ name: string; influence: number } } }).morphTargetManager if (!
manager) continue for (let i = 0; i < manager.numTargets; i++) {
const target = manager.getTarget(i) if (target.name === name)
target.influence = value } } } private
_buildNameTag(): void { // Create a plane above the avatar that always
faces the camera const plane =
MeshBuilder.CreatePlane( 'nametag_plane', { width: 2, height: 0.4
}, this._scene ) plane.parent = this.rootMesh
plane.position.y = 2.4 plane.billboardMode =
BillboardMode.BILLBOARDMODE_ALL const texture =
AdvancedDynamicTexture.CreateForMesh(plane, 512, 128) const label = new
TextBlock('nametag_text', this._user.username) label.color = '#ffffff'
label.fontSize = 48 label.fontFamily = 'Calibri'
texture.addControl(label) this._nameTagMesh = plane } private
_playAnimation(name: string, loop: boolean): void { if
(this._currentAnim) this._currentAnim.stop() const anim =
this._animations.get(name) if (anim) { anim.start(loop, 1.0,
anim.from, anim.to, false) this._currentAnim = anim } }
private _registerRenderLoop(): void { this._scene.registerBeforeRender(()
=> { if (!this._targetPosition || !this._isMoving) return const
current = this.rootMesh.position const dir =
this._targetPosition.subtract(current) const dist = dir.length()
const dt = this._scene.getEngine().getDeltaTime() / 1000 if (dist <
0.05) { this.rootMesh.position.copyFrom(this._targetPosition)
this._isMoving = false this._playAnimation('idle', true) //
Confirm final position to server
SocketService.getInstance().emit('player:move', { roomId:
window.__havenRoomId, x: this._targetPosition.x, y:
this._targetPosition.y, z: this._targetPosition.z, rotY:
this.rootMesh.rotation.y, }) return } const step
= dir.normalize().scale(this._moveSpeed * dt)
this.rootMesh.position.addInPlace(step) // Rotate to face movement
direction const angle = Math.atan2(dir.x, dir.z)
this.rootMesh.rotation.y = angle } } /** Move avatar to a target
world position. */ public moveTo(target: Vector3): void
{ this._targetPosition = target.clone() this._targetPosition.y =
0 // Always walk on ground plane this._isMoving = true
this._playAnimation('walk', true) } public applyAvatarData(data:
AvatarData): void { this._applyMaterials(data) } public dispose():
void { this._meshes.forEach(m => m.dispose())
this._nameTagMesh?.dispose() this.rootMesh.dispose() } }

```

12 — RemoteAvatar.ts: Other Players' Avatars

The **RemoteAvatar** mirrors the AvatarController but is driven by incoming socket events rather than user input. It uses linear interpolation to smooth position updates and supports floating chat bubbles. Create `apps/client/src/world/RemoteAvatar.ts`:

```

import { Scene, TransformNode, AbstractMesh, AnimationGroup,
Vector3, Color3, StandardMaterial, MeshBuilder, BillboardMode,
SceneLoader, } from '@babylonjs/core' import { AdvancedDynamicTexture,
TextBlock, Rectangle } from '@babylonjs/gui' export class RemoteAvatar
{   public rootMesh!: TransformNode   private _scene: Scene   private
_userId: string   private _username: string   private _meshes: AbstractMesh[]
= []   private _animations: Map = new Map()   private _currentAnim:
AnimationGroup | null = null   // Lerp state   private _lerpTarget: Vector3
= Vector3.Zero()   private _lerpTargetRotY = 0   private _lerpSpeed = 8 //
higher = snappier, lower = smoother   // Chat bubble   private _chatPlane:
AbstractMesh | null = null   private _chatTexture: AdvancedDynamicTexture |
null = null   private _chatTimeout: ReturnType | null = null   private
_isMoving = false   constructor(scene: Scene, userId: string, username:
string) {       this._scene = scene       this._userId = userId
this._username = username   }   public async init(): Promise {       try {
await this._loadGLB()       } catch {       this._buildPlaceholder()       }
this._buildNameTag()       this._buildChatBubble()
this._registerRenderLoop()   }   private async _loadGLB(): Promise
{       const result = await SceneLoader.ImportMeshAsync('', '',
'/assets/avatars/base_avatar.glb', this._scene)       this.rootMesh = new
TransformNode(`remote_${this._userId}_root`, this._scene)
result.meshes[0].parent = this.rootMesh       this._meshes = result.meshes as
AbstractMesh[]       for (const ag of result.animationGroups)
{           this._animations.set(ag.name, ag)           ag.stop()       }
this._playAnimation('idle', true)   }   private _buildPlaceholder(): void {
this.rootMesh = new TransformNode(`remote_${this._userId}_root`, this._scene)
const body = MeshBuilder.CreateCapsule(       `remote_${this._userId}_body`,
{ radius: 0.25, height: 1.7, tessellation: 8 },       this._scene   )
body.parent = this.rootMesh       body.position.y = 0.85       const head =
MeshBuilder.CreateSphere(       `remote_${this._userId}_head`,
{ diameter: 0.4, segments: 8 },       this._scene   )       head.parent =
this.rootMesh       head.position.y = 1.95       // Each remote avatar gets a
unique hue based on userId hash       const hue = (this._userId.charCodeAt(0) *
37 + this._userId.charCodeAt(1) * 17) % 360       const mat = new
StandardMaterial(`remote_${this._userId}_mat`, this._scene)
mat.diffuseColor = Color3.FromHSV(hue, 0.6, 0.9)       body.material = mat
head.material = mat       this._meshes = [body, head]   }   private
_buildNameTag(): void {       const plane = MeshBuilder.CreatePlane(`remote_${
this._userId}_nametag`, { width: 2, height: 0.4 }, this._scene)
plane.parent = this.rootMesh       plane.position.y = 2.4
plane.billboardMode = BillboardMode.BILLBOARDMODE_ALL       const tex =
AdvancedDynamicTexture.CreateForMesh(plane, 512, 128)       const label = new
TextBlock('', this._username)       label.color = '#ffffcc'       label.fontSize
= 44       tex.addControl(label)       this._meshes.push(plane)   }   private
_buildChatBubble(): void {       const plane =
MeshBuilder.CreatePlane(`remote_${this._userId}_chat`, { width: 3, height:
0.7 }, this._scene)       plane.parent = this.rootMesh       plane.position.y =
3.0       plane.billboardMode = BillboardMode.BILLBOARDMODE_ALL
plane.isVisible = false       const tex =
AdvancedDynamicTexture.CreateForMesh(plane, 768, 192)       const bg = new
Rectangle('chatbg')       bg.background = '#000000cc'       bg.color = '#ffffff'
bg.thickness = 1       bg.cornerRadius = 8       tex.addControl(bg)       const
text = new TextBlock('chattext', '')       text.color = '#ffffff'
text.fontSize = 36       text.textWrapping = true       bg.addControl(text)
this._chatPlane = plane       this._chatTexture = tex   }   /** Show a chat
bubble above the avatar for 4 seconds. */   public showChatBubble(message:
string): void {       if (!this._chatPlane || !this._chatTexture) return
const text = this._chatTexture.getControlByName('chattext') as TextBlock
if (text) text.text = message       this._chatPlane.isVisible = true       if
(this._chatTimeout) clearTimeout(this._chatTimeout)       this._chatTimeout =

```

```

setTimeout(() => {      if (this._chatPlane) this._chatPlane.isVisible =
false      }, 4000)  }   private _playAnimation(name: string, loop:
boolean): void {      if (this._currentAnim) this._currentAnim.stop()
const anim = this._animations.get(name)      if (anim)
{      anim.start(loop, 1.0, anim.from, anim.to, false)
this._currentAnim = anim      }   }   private _registerRenderLoop(): void {
this._scene.registerBeforeRender(() => {      if (!this.rootMesh) return
const dt = this._scene.getEngine().getDeltaTime() / 1000      const alpha =
Math.min(1, this._lerpSpeed * dt)      const prev =
this.rootMesh.position.clone()      this.rootMesh.position =
Vector3.Lerp(this.rootMesh.position, this._lerpTarget, alpha)      const
moved = Vector3.Distance(prev, this.rootMesh.position)      if (moved > 0.02
&& !this._isMoving) {      this._isMoving = true
this._playAnimation('walk', true)      } else if (moved < 0.01 &&
this._isMoving) {      this._isMoving = false
this._playAnimation('idle', true)      }      // Lerp rotation
this.rootMesh.rotation.y += (this._lerpTargetRotY - this.rootMesh.rotation.y)
* alpha    })    }    /** Called when a player:move event arrives from the
server. */    public moveTo(target: Vector3, rotY: number): void
{      this._lerpTarget = target.clone()      this._lerpTarget.y = 0
this._lerpTargetRotY = rotY    }    public setPosition(pos: Vector3): void {
if (!this.rootMesh) return      this.rootMesh.position.copyFrom(pos)
this._lerpTarget = pos.clone()    }    public applyAvatarData(data: unknown):
void {      // Applied in Part 5B when real GLB + morph targets are available
console.log(`[RemoteAvatar] Avatar data update for ${this._userId}:`, data)
}    public get userId(): string { return this._userId }    public dispose():
void {      if (this._chatTimeout) clearTimeout(this._chatTimeout)
this._meshes.forEach(m => m.dispose())      this.rootMesh?.dispose()    } }

```

13 — InputController.ts: Click-to-Move Input

The **InputController** handles pointer events, performs ray-casting from the camera through the click point onto the scene, and triggers avatar movement when the ray intersects a tagged walkable surface. Create

apps/client/src/engine/InputController.ts:

```

import { Scene, PointerEventTypes, Ray, Vector3, Tags,
AbstractMesh, } from '@babylonjs/core' import type { AvatarController } from
'../world/AvatarController' export class InputController { private _scene:
Scene private _avatar: AvatarController private _enabled = false
constructor(scene: Scene, avatar: AvatarController) {      this._scene = scene
this._avatar = avatar    }    public enable(): void {      if (this._enabled)
return      this._enabled = true
this._scene.onPointerObservable.add(this._onPointer)    }    public disable():
void {      this._enabled = false
this._scene.onPointerObservable.removeCallback(this._onPointer)    }
private _onPointer = (pointerInfo: { type: number; pickInfo: { hit: boolean;
pickedMesh: AbstractMesh | null; pickedPoint: Vector3 | null } | null }):
void => {      if (pointerInfo.type !== PointerEventTypes.POINTERUP) return
if (!pointerInfo.pickInfo?.hit) return      const pickedMesh =
pointerInfo.pickInfo.pickedMesh      const pickedPoint =
pointerInfo.pickInfo.pickedPoint      if (!pickedMesh || !pickedPoint) return
// Only move if the clicked mesh is tagged as walkable      if (!
Tags.HasTags(pickedMesh) || !Tags.MatchesQuery(pickedMesh, 'walkable'))

```

```

return      // Move the local avatar to the clicked point on the floor
this._avatar.moveTo(new Vector3(pickedPoint.x, 0, pickedPoint.z)) } /**
 * Manual ray-cast utility – used when you need to cast from a specific
 * screen-space point rather than the last pointer event. */ public
castRayFromScreen(screenX: number, screenY: number): Vector3 | null
{      const ray = this._scene.createPickingRay(      screenX,      screenY,
null,      this._scene.activeCamera      )      const hit =
this._scene.pickWithRay(ray, (mesh) =>      Tags.HasTags(mesh) &&
Tags.MatchesQuery(mesh, 'walkable')      )      return hit?.pickedPoint ?? null
} }

```

14 — ChatOverlay.ts: HTML Chat System

The chat overlay is implemented as a **pure HTML/CSS element** positioned over the canvas using `position: fixed`. It does not use Babylon GUI — this ensures it is accessible, scrollable, and easy to style without 3D engine overhead. Create `apps/client/src/ui/ChatOverlay.ts`:

```

import { SocketService } from '../services/SocketService' import
{ AuthService } from '../services/AuthService' export class ChatOverlay
{      private _container: HTMLDivElement      private _log: HTMLDivElement
private _input: HTMLInputElement      private _sendBtn: HTMLButtonElement
private _roomId: string      private _socket = SocketService.getInstance()
constructor(roomId: string) {      this._roomId = roomId      this._container =
this._buildDOM()      document.body.appendChild(this._container)
this._wireEvents()      this._listenSocket()      }      private _buildDOM():
HTMLDivElement {      const container = document.createElement('div')
container.id = 'haven-chat'      container.style.cssText = `      position:
fixed;      bottom: 16px;      left: 16px;      width: 320px;
background: rgba(0,0,0,0.72);      border: 1px solid rgba(255,255,255,0.18);
font-family: Calibri, sans-serif;      font-size: 13px;      color:
#ffffff;      z-index: 9999;      display: flex;      flex-direction:
column;      `      // Message log      const log =
document.createElement('div')      log.id = 'haven-chat-log'
log.style.cssText = `      height: 180px;      overflow-y: auto;
padding: 8px 10px;      line-height: 1.5;      display: flex;      flex-
direction: column;      gap: 2px;      `      this._log = log      // Input
row      const inputRow = document.createElement('div')
inputRow.style.cssText = `      display: flex;      border-top: 1px solid
rgba(255,255,255,0.15);      padding: 6px 8px;      gap: 6px;
const input = document.createElement('input')      input.type = 'text'
input.placeholder = 'Say something...'      input.maxLength = 200
input.style.cssText = `      flex: 1;      background:
rgba(255,255,255,0.1);      border: 1px solid rgba(255,255,255,0.2);
color: #ffffff;      padding: 4px 8px;      font-size: 13px;      outline:
none;      font-family: Calibri, sans-serif;      `      this._input = input
const sendBtn = document.createElement('button')      sendBtn.textContent =
'Send'      sendBtn.style.cssText = `      background: #2b5797;      color:
#ffffff;      border: none;      padding: 4px 12px;      cursor: pointer;
font-family: Calibri, sans-serif;      font-size: 13px;
this._sendBtn = sendBtn      inputRow.appendChild(input)
inputRow.appendChild(sendBtn)      container.appendChild(log)
container.appendChild(inputRow)      return container      }      private

```

```

_wireEvents(): void {      const send = (): void => {      const msg =
this._input.value.trim()    if (!msg) return
this._socket.emit('chat:send', {      roomId: this._roomId,
message: msg,      })      this._input.value = ''      }
this._sendBtn.addEventListener('click', send)
this._input.addEventListener('keydown', (e) => {      if (e.key === 'Enter')
send()      })      }      private _listenSocket(): void
{      this._socket.on('chat:message', (data: { userId: string; username:
string; message: string }) => {      this._appendMessage(data.username,
data.message)      })      }      private _appendMessage(username: string, message:
string): void {      const line = document.createElement('div')
line.style.cssText = 'word-break: break-word;'      const nameSpan =
document.createElement('span')      nameSpan.style.cssText = 'font-weight:
bold; color: #88bbff;'      nameSpan.textContent = username + ': '      const
msgSpan = document.createElement('span')      msgSpan.textContent =
message      // Server already applied bad-words filter
line.appendChild(nameSpan)      line.appendChild(msgSpan)
this._log.appendChild(line)      // Auto-scroll to bottom
this._log.scrollTop = this._log.scrollHeight      }      public
appendSystemMessage(text: string): void {      const line =
document.createElement('div')      line.style.cssText = 'color: #aaaaaa; font-
style: italic;'      line.textContent = text      this._log.appendChild(line)
this._log.scrollTop = this._log.scrollHeight      }      public dispose(): void {
this._socket.off('chat:message')      this._container.remove()      } }

```

15 — Updated main.ts Entry Point

The new `main.ts` bootstraps the engine, registers all scenes, checks auth state, and navigates to the appropriate scene. Replace `apps/client/src/main.ts` entirely:

```

import './style.css' import { HavenEngine } from './engine/HavenEngine'
import { SceneManager } from './engine/SceneManager' import { AuthService }
from './services/AuthService' import { SocketService } from
'./services/SocketService' // — CANVAS SETUP

const root =
document.getElementById('game-root') if (!root) throw new Error('[main]
#game-root not found in DOM.') const canvas =
document.createElement('canvas') canvas.id = 'haven-canvas'
canvas.style.cssText = 'width:100%;height:100%;display:block;touch-
action:none;' root.appendChild(canvas) // — LOADING SCREEN

const
loadingScreen = document.getElementById('haven-loading') const hideLoading =
(): void => {      if (loadingScreen) loadingScreen.style.display = 'none' } //
— BOOTSTRAP —

async function bootstrap(): Promise {      // Initialize engine (creates
Babylon.js Engine + Havok + Inspector in dev)      const haven = await
HavenEngine.getInstance(canvas)      const scenes = SceneManager.getInstance()
const auth = AuthService      // Register scene factories
scenes.register('login', async (_haven) => {      const { createLoginScene } =
await import('./scenes/LoginScene')      return createLoginScene(_haven)      })
scenes.register('lobby', async (_haven) => {      const { createLobbyScene } =
await import('./scenes/LobbyScene')      return createLobbyScene(_haven)      })
scenes.register('room', async (_haven) => {      const roomId = (window as
unknown as Record).__havenRoomId ?? 'main'      const { createRoomScene } =
await import('./scenes/RoomScene')      return createRoomScene(_haven, roomId)

```

```

}) // Route based on auth state  const token = auth.getToken()  if
(token && auth.isTokenValid(token)) { // Connect socket before entering
lobby  SocketService.getInstance().connect(token)  await
scenes.switchTo('lobby')  } else {  await scenes.switchTo('login')  }
hideLoading() } // — GLOBAL ERROR BOUNDARY

window.addEventListener('unhandledrejection', (event) =>
{  console.error('[HavenWorld] Unhandled rejection:', event.reason)  //
Sentry: Sentry.captureException(event.reason) })
window.addEventListener('error', (event) => {  console.error('[HavenWorld]
Global error:', event.error)  // Sentry:
Sentry.captureException(event.error) }) bootstrap().catch((err) =>
{  console.error('[HavenWorld] Bootstrap failed:', err) })

```

16 — Updated index.html

Replace `apps/client/index.html` with the following. The key changes from Phase 1: Phaser-specific styles removed, full-screen canvas target added, loading screen div added, and HTML overlay panels defined for chat and HUD positioned over the canvas.

```

<!DOCTYPE html> <html lang="en"> <head>  <meta charset="UTF-8" />  <meta
name="viewport" content="width=device-width, initial-scale=1.0" />
<title>HavenWorld</title>  <style>    *, *::before, *::after { box-sizing:
border-box; margin: 0; padding: 0; }    html, body {      width: 100%;
height: 100%;      overflow: hidden;      background: #0d0d0d;      font-
family: Calibri, sans-serif;    }    #game-root {      width: 100vw;
height: 100vh;      position: relative;    }    #haven-canvas
{      position: absolute;      top: 0; left: 0;      width: 100%; height:
100%;      display: block;      touch-action: none;    }    /* —
Loading screen — */    #haven-loading {      position: fixed;      top:
0; left: 0;      width: 100%; height: 100%;      background: #0d0d0d;
display: flex;      flex-direction: column;      align-items: center;
justify-content: center;      z-index: 99999;      color: #ffffff;
font-family: Calibri, sans-serif;    }    #haven-loading h2 { font-size:
22pt; margin-bottom: 18px; letter-spacing: 2px; }    .haven-spinner
{      width: 48px; height: 48px;      border: 4px solid
rgba(255,255,255,0.15);      border-top-color: #2b5797;      border-radius:
50%;      animation: spin 0.9s linear infinite;    }    @keyframes spin
{ to { transform: rotate(360deg); } }    /* — HUD overlay (top-right) —
*/    #haven-hud {      position: fixed;      top: 16px; right: 16px;
z-index: 9000;      color: #ffffff;      font-size: 12pt;      text-
shadow: 1px 1px 2px #000000;      pointer-events: none;    }  </style>
</head> <body>  <!-- Loading screen (hidden after engine ready) -->  <div
id="haven-loading">    <h2>HAVENWORLD</h2>    <div class="haven-
spinner"></div>  </div>  <!-- 3D canvas root -->  <div
id="game-root"></div>  <!-- HUD overlay -->  <div id="haven-hud"></div>
<!-- Chat overlay is injected by ChatOverlay.ts -->  <!-- Avatar customizer
overlay is injected by AvatarCustomizer.ts (Part 5B) -->  <script
type="module" src="/src/main.ts"></script> </body> </html>

```

17 — Updating packages/shared for 3D

The shared package requires only **additive changes** — no existing types or constants are removed. The 2D `PlayerPosition` type is kept for reference during migration; the new `PlayerPosition3D` type is added alongside it. Open `packages/shared/src/index.ts` and add the following to the existing file:

```
// — EXISTING (unchanged from Phase 1)
export const SOCKET_EVENTS =
{
  PLAYER_JOIN: 'player:join',
  PLAYER_LEAVE: 'player:leave',
  PLAYER_MOVE: 'player:move',
  AVATAR_UPDATE: 'avatar:update',
  CHAT_SEND: 'chat:send',
  CHAT_MESSAGE: 'chat:message',
  ROOM_JOIN: 'room:join',
  ROOM_LEAVE: 'room:leave',
} as const /** @deprecated Phase 1 — 2D tile coordinates. Use PlayerPosition3D in Phase 2+. */
export interface PlayerPosition2D {
  x: number // tile column
  y: number // tile row
}

// — NEW — Phase 2 (3D world space)
/** * 3D world-space position used by Babylon.js. * Units are meters. Y is the vertical axis (up). * rotY is the avatar's Y-axis rotation in radians. * * Migration note: Phase 1 used tile coordinates (x=col, y=row). * Phase 2 uses world-space meters (x, z for horizontal, y for vertical height). * Typical floor level is y=0. Avatar starts at (0, 0, 0). */
export interface PlayerPosition3D {
  x: number // world-space meters, horizontal (left/right)
  y: number // world-space meters, vertical (height — usually 0 for ground)
  z: number // world-space meters, horizontal (forward/back)
  rotY: number // avatar facing direction in radians (Y-axis rotation)
}

/** player:move socket payload — updated for Phase 2 */
export interface PlayerMovePayload {
  roomId: string
  position: PlayerPosition3D
}

/** player:join socket payload */
export interface PlayerJoinPayload {
  userId: string
  username: string
  position: PlayerPosition3D
  avatarData: AvatarCustomizationData
}

/** Avatar customization data shared between client and server */
export interface AvatarCustomizationData {
  skinColor: string // hex e.g. "#F5CBA7"
  hairColor: string // hex e.g. "#3B2314"
  eyeColor: string // hex e.g. "#3B82F6"
  bodyMorphs?: Record<string, number> // morph target name → 0..1 influence
}
```

18 — Updating the Server for 3D Positions

The server changes for Phase 2 are **minimal by design**. The Socket.io event names stay identical. Only the shape of the `player:move` payload changes. No Prisma schema changes are needed — player positions are stored in Redis in-memory (from Part 4B), not in PostgreSQL.

18.1 — Update `player:move` Handler

Open `apps/server/src/socket/handlers.ts`. Find the existing `player:move` handler block and apply this diff:

```
// — BEFORE (Phase 1 – 2D tile coords)
socket.on('player:move', async (data: { roomId: string; x: number; y: number }) => { const { roomId, x, y } = data
  await roomManager.updatePosition(socket.data.userId, roomId, { x, y })
  socket.to(roomId).emit('player:move', {      userId: socket.data.userId,
x,      y,    }) }) // — AFTER (Phase 2 – 3D world-space)
socket.on('player:move', async (data: { roomId: string; x: number; y: number; z: number; rotY: number }) => {
  const { roomId, x, y, z, rotY } = data // Basic server-side sanity
  clamp (prevent teleport exploits) const MAX_COORD = 200 const cx = Math.max(-MAX_COORD, Math.min(MAX_COORD, x))
  const cy = Math.max(0, Math.min(10, y)) // y should be near 0 (floor) const cz = Math.max(-MAX_COORD, Math.min(MAX_COORD, z))
  const cRotY = rotY % (Math.PI * 2)
  await roomManager.updatePosition(socket.data.userId, roomId, { x: cx, y: cy, z: cz, rotY: cRotY }) // Broadcast to everyone else in the room (not sender)
  socket.to(roomId).emit('player:move', {      userId: socket.data.userId,      x: cx,      y: cy,      z: cz,      rotY: cRotY,    }) }) }
```

18.2 — Update RoomManager In-Memory State

In `apps/server/src/rooms/RoomManager.ts`, update the player state type stored in Redis:

```
// — BEFORE
interface PlayerState {  userId: string  username: string  x: number  y: number } // — AFTER
interface PlayerState {  userId: string  username: string  x: number  y: number  z: number  rotY: number  avatarData?: Record<string, unknown> }
```

✓ No Prisma Migration Needed

Player positions are ephemeral in-memory state stored in Redis (per room), not persisted to PostgreSQL. The Prisma schema for `chat_messages`, `users`, and all other tables is completely unchanged. No `prisma migrate` command is needed for Phase 2A.

19 — Asset Placeholders: Getting Started Without 3D Art

Most developers starting Phase 2 will not yet have production-ready 3D models. This section provides complete Babylon.js code to stand in for real GLB files so you can develop and test all engine systems immediately.

19.1 — Placeholder Room (No GLB Required)

Create `apps/client/src/world/PlaceholderRoom.ts`:

```
import { Scene, MeshBuilder, StandardMaterial, Color3, Vector3, Tags, PointLight, } from '@babylonjs/core' /** * Creates a simple placeholder room using Babylon MeshBuilder primitives. * Used when no GLB room file is available. Replace with a real GLB in Part 5B. * Room dimensions: 30m x 30m floor, 4m walls. */ export function createPlaceholderRoom(scene: Scene): void { // — FLOOR

    const floor = MeshBuilder.CreateGround('Floor', { width: 30, height: 30, subdivisions: 1 }, scene)
    const floorMat = new StandardMaterial('floor_mat', scene)
    floorMat.diffuseColor = new Color3(0.72, 0.68, 0.60)
    floor.material = floorMat
    floor.isPickable = true
    Tags.AddTagsTo(floor, 'walkable') // — WALLS

    const wallMat = new StandardMaterial('wall_mat', scene)
    wallMat.diffuseColor = new Color3(0.88, 0.85, 0.80)
    const wallDefs = [
        { name: 'wall_north', pos: new Vector3(0, 2, -15), rot: 0 },
        { name: 'wall_south', pos: new Vector3(0, 2, 15), rot: 0 },
        { name: 'wall_east', pos: new Vector3(15, 2, 0), rot: Math.PI / 2 },
        { name: 'wall_west', pos: new Vector3(-15, 2, 0), rot: Math.PI / 2 },
    ]
    for (const def of wallDefs) {
        const wall = MeshBuilder.CreatePlane(def.name, { width: 30, height: 4 }, scene)
        wall.position.copyFrom(def.pos)
        wall.rotation.y = def.rot
        wall.material = wallMat
        wall.isPickable = false
        wall.backFaceCulling = false
    } // — AMBIENT POINT LIGHT

    const roomLight = new PointLight('room_light', new Vector3(0, 5, 0), scene)
    roomLight.intensity = 0.5
    console.log('[PlaceholderRoom] Created 30x30m placeholder room.') }
```

19.2 — Placeholder Avatar (No GLB Required)

The `AvatarController._buildPlaceholder()` method (Section 11) already handles this case automatically — it builds a capsule + sphere avatar using `MeshBuilder` and assigns a material with the user's configured skin color. No separate file is needed; the fallback is built into the controller itself.

19.3 — Free Placeholder GLB Sources

Source	URL	License	Best Search Terms
Sketchfab	sketchfab.com/search?q=room+interior&type=models&features=downloadable&license=cc-by	CC-BY 4.0	"room interior", "cartoon living room"
KhronosGroup GLTF Samples	github.com/KhronosGroup/glTF-Sample-	Various (CC-BY, MIT)	Reference GLB/GLTF test files

Source	URL	License	Best Search Terms
	Models		
Babylon.js CDN	cdn.babylonjs.com/Assets/	Apache-2.0	Used in Babylon Playground examples
Mixamo (characters)	mixamo.com	Adobe Standard (free tier)	Humanoid rigged characters with animations
Poly Haven	polyhaven.com/models	CC0 (Public Domain)	Furniture, props, room assets

⚠ **Poly Haven is CC0 — Best Choice**

Poly Haven assets are **CC0 (public domain)** — no attribution required and fully compatible with GPL-3.0. Use Poly Haven assets for any furniture or props you include in HavenWorld's default room. Sketchfab CC-BY assets require attribution in your project credits.

20 — Running and Testing Phase 2 (Part A)

20.1 — Development Server

MAC Start the full development stack:

```
cd ~/projects/HavenWorld # Terminal 1 – Server (unchanged from Phase 1) pnpm --filter server dev # Terminal 2 – Client (Babylon.js, Vite 6) pnpm --filter client dev # Open http://localhost:5173
```

LINUX Same commands — no differences from Mac for the development run.

20.2 — Verification Checklist

1. Canvas renders — the placeholder room (flat grey plane + white walls) is visible. No black screen.
2. Your placeholder avatar capsule appears in the center of the room.

3. Click anywhere on the grey floor plane — the avatar capsule moves toward the click point.
4. Browser console shows `Babylon.js 7.x.x` initialized with no TypeScript errors.
5. Press **Ctrl+I** (Mac: **Cmd+I**) — the Babylon Inspector panel slides open from the right, showing the scene graph with all meshes listed.
6. Open a second browser tab, log in as a different test user, and join the same room ID.
7. In Tab 1: click to move. In Tab 2: a second avatar capsule (different color) lerps smoothly to the new position within ~100ms.
8. Type a message in the Chat Overlay (bottom-left) and press Send — the message appears in the log in both tabs, and a floating bubble appears above the sending avatar in Tab 2.
9. Close Tab 2 — in Tab 1, the remote avatar capsule disappears (player:leave event fired correctly).

20.3 — Production Build

MAC

```
pnpm --filter client build # Output in apps/client/dist/ # Check bundle sizes: du -sh apps/client/dist/assets/*.js | sort -h
```

⚠ Bundle Size Warning

Babylon.js is ~1.4MB minified before gzip. With tree-shaking and the `manualChunks` split configured in Section 5, expect the Babylon chunk to be approximately **600-900KB gzipped**. Cloudflare Pages compresses assets automatically. If the uncompressed `babylon.js` chunk exceeds **5MB**, check that you are not accidentally importing the entire `@babylonjs/core` namespace — use named imports only (e.g., `import { Scene, Engine } from '@babylonjs/core'`).

21 — CI/CD Notes

21.1 — Havok WASM File Handling

The Havok physics WASM binary (`HavokPhysics.js.wasm`) is distributed with the `@babylonjs/havok` package but **must also be present in the `public/` folder** for Cloudflare Pages to serve it with the correct MIME type. Copy it during your build step:

```
# Add to apps/client/package.json scripts: "prebuild": "cp
node_modules/@babylonjs/havok/lib/esm/HavokPhysics.js.wasm public/"
```

Add the following to your `.gitignore` exclusion list to ensure the WASM file IS committed (override the usual `node_modules` ignore):

```
# In root .gitignore – allow the Havok WASM in public/
!apps/client/public/HavokPhysics.js.wasm
```

21.2 — Cloudflare Pages Headers File

Create `apps/client/public/_headers` to set the correct MIME type and required security headers for WASM + `SharedArrayBuffer`:

```
/HavokPhysics.js.wasm Content-Type: application/wasm /* Cross-Origin-
Opener-Policy: same-origin Cross-Origin-Embedder-Policy: require-corp
```

⚠ COOP/COEP Headers Are Required

The `Cross-Origin-Opener-Policy: same-origin` and `Cross-Origin-Embedder-Policy: require-corp` headers are **required** for `SharedArrayBuffer` to be available, which Havok WASM depends on. Without these headers, Havok physics will silently fall back to disabled. The `_headers` file in Cloudflare Pages `public/` folder applies these automatically on deploy.

21.3 — Cloudflare Pages Build Settings (Unchanged)

Setting	Value
Build command	<code>pnpm --filter client build</code>
Build output directory	<code>apps/client/dist</code>
Node.js version	22

Setting	Value
Root directory	(repository root)

22 — Troubleshooting Phase 2A

#	Symptom	Root Cause	Fix
1	SharedArrayBuffer is not defined in console	Missing COOP/COEP headers — Havok WASM requires cross-origin isolation	Add Cross-Origin-Opener-Policy: same-origin and Cross-Origin-Embedder-Policy: require-corp to your _headers file (Section 21.2). For local dev, add these headers to Vite's server.headers config.
2	GLB file 404 / "room not found"	GLB placed in src/ instead of public/	All static assets served by Vite at runtime must be in apps/client/public/. Place GLBs at public/assets/rooms/main.glb. Verify by navigating directly to http://localhost:5173/assets/rooms/main.glb.
3	WASM fails to load in production	assetsInlineLimit not set — Vite inlines small files as base64, breaking WASM	Set assetsInlineLimit : 0 in vite.config.ts (already in Section 5 config). Rebuild and redeploy.
4	Canvas is completely black	No lights in the scene, or scene has no active	Add a HemisphericLight first before

#	Symptom	Root Cause	Fix
		camera	debugging materials. Verify <code>scene.activeCamera</code> is not null. Check the engine render loop is running with <code>engine.isDisposed</code> .
5	Avatar does not move when floor is clicked	Floor mesh name mismatch — <code>InputController</code> checks for walkable tag	Open the Inspector (Ctrl+I), click the floor mesh, and verify the Tags property contains walkable. If using a GLB room, ensure the floor mesh name starts with <code>Walkable_</code> or is exactly <code>Floor</code> . For placeholder room, the <code>createPlaceholderRoom</code> function already applies the tag correctly.
6	Remote avatars teleport instead of lerp	<code>_lerpSpeed</code> too high, or position updates are being applied directly instead of to <code>_lerpTarget</code>	In <code>RemoteAvatar.ts</code> , verify the render loop updates <code>rootMesh.position</code> via <code>Vector3.Lerp()</code> with <code>_lerpTarget</code> , not by calling <code>rootMesh.position.copyFrom()</code> directly. Lower <code>_lerpSpeed</code> from 8 to 4 for smoother movement over slow connections.
7	Babylon Inspector does not open (Ctrl+I does nothing)	<code>@babylonjs/inspector</code> is not in <code>devDependencies</code> , or the dynamic import is failing silently	Verify <code>@babylonjs/inspector</code> is in <code>apps/client/package.json</code> under <code>devDependencies</code> .

#	Symptom	Root Cause	Fix
			Check the browser console for [HavenEngine] Inspector loaded message. The dynamic import only runs when <code>import.meta.env.DEV</code> is true — confirm you are running vite dev, not the built output.
8	TypeScript errors: Cannot find module '@babylonjs/core'	pnpm workspace not hoisting packages correctly	Run <code>pnpm install</code> from the monorepo root. Confirm <code>apps/client/node_modules/@babylonjs</code> exists (or is symlinked). Check that <code>packages/shared</code> does not accidentally pull a conflicting Babylon version.

What's Next: Part 5B

Part 5A has established the full 3D engine foundation: Babylon.js is wired up, scenes transition cleanly, click-to-move works, Socket.io multiplayer sync is verified in 3D space, and the chat overlay is functional. All of this works with placeholder geometry. Part 5B replaces the placeholders with real production assets and adds the avatar customization pipeline.

Part 5B Coverage

10. **Blender → GLB Pipeline** — Creating optimized room geometry in Blender 5.1, exporting to GLB, applying Draco mesh compression and KTX2 texture

compression with gltf-transform CLI, and committing to `public/assets/rooms/`.

11. **Real Rigged Humanoid Avatar** — Importing a fully-rigged character GLB (Mixamo or custom), confirming bone hierarchy, and integrating with `AvatarController` to replace the placeholder capsule.
12. **Mixamo Animations** — Downloading idle, walk, sit, wave, and dance animations from Mixamo, retargeting them to the HavenWorld skeleton in Blender, and packing them into the base avatar GLB.
13. **Morph Targets for Body Customization** — Authoring body shape morph targets in Blender (height, weight, proportions), exporting with morph data, and hooking them into `AvatarController._applyMaterials()`.
14. **3D Avatar Customizer UI** — Building the `AvatarCustomizer.ts` HTML overlay panel with sliders for body morphs, color pickers for skin/hair/eye, and live preview.
15. **Furniture Placement System** — Implementing `FurnitureManager.ts` for drag-and-drop room furniture using Babylon pointer drag behaviors, persistence to the server via Socket.io, and per-room save state in Redis/Postgres.
16. **Personal Room Editor** — Player-owned rooms with furniture load/save from the Prisma `rooms` table, room switching from lobby, and room ownership permission checks.

Before You Commit Part 5A

Use descriptive, conventional-commit-style messages. Suggested commit sequence:

```
feat(client): add Babylon.js engine singleton and Havok physics init
feat(client): add SceneManager scene registry and transition system
```

feat(client): add RoomLoader GLB import with walkable mesh tagging

feat(client): add RoomScene isometric camera, lights, socket wiring

feat(client): add AvatarController with placeholder capsule fallback

feat(client): add RemoteAvatar with lerp and chat bubble

feat(client): add InputController click-to-move floor ray-cast

feat(client): add ChatOverlay HTML fixed panel with socket wiring

feat(client): update main.ts bootstrap and index.html for Babylon

feat(shared): add PlayerPosition3D interface and PlayerMovePayload

fix(server): update player:move handler to accept x/y/z/rotY

chore(client): add Cloudflare Pages _headers for WASM COOP/COEP

Useful Links for Phase 2 Development

Resource	URL	Use For
Babylon.js Playground	playground.babylonjs.com	Testing Babylon code snippets in isolation before adding to project
Babylon.js Docs	doc.babylonjs.com	API reference, tutorials, WebXR guides
Babylon.js Forum	forum.babylonjs.com	Community Q&A — very active, core team responds
Apache-2.0 / GPL-3.0 Compatibility	gnu.org/licenses/license-list.html#apache2	Confirm Apache-2.0 is compatible as a GPL-3.0 dependency (it is)
HavenWorld Repo	github.com/dboone323/HavenWorld	Project source code, issues, CI runs

Apache-2.0 and GPL-3.0 Compatibility

The Free Software Foundation confirms that **Apache-2.0 is compatible with GPL-3.0** when used as a dependency in a GPL-3.0 project. Babylon.js (Apache-2.0) can be legally included in HavenWorld (GPL-3.0) without license conflict. The combined work must be distributed under GPL-3.0 terms, which is already the case for HavenWorld. See gnu.org/licenses/license-list.html#apache2 for the official FSF statement.